

knowyourpit

App Knowledge Document — Agent Handoff Reference

Generated May 29, 2026

Table of Contents

- 1. Project Overview
- 2. Database Schema
- 3. API Reference
- 4. Mobile Screen Map
- 5. API Server Routes
- 6. Environment Variables
- 7. Conventions & Preferences

Project Overview

knowyourpit

Overview

knowyourpit is an AI-powered BBQ planning and management application. It offers tools for managing grill profiles, logging cook sessions, and providing AI-driven cook plans and time predictions. The app also features temperature monitoring, recipe management, and personalized pit master coaching.

User Preferences

- All `eas` and `expo` commands must be run from `artifacts/knowyourpit/`, never from the workspace root.
- Do not delete the disabled Apple Watch companion app code; it is the starting point for future modernization work.
- Never auto-initiate an EAS build. Always ask the user for confirmation before queuing any build.

Mobile UI Conventions

Keyboard-safe modals

Every modal or bottom sheet that contains a `TextInput` **must** use `AppKeyboardAvoidingView` instead of the raw React Native `KeyboardAvoidingView`. The component lives at `artifacts/knowyourpit/components/AppKeyboardAvoidingView.tsx` and has the correct cross-platform behavior baked in (`"padding"` on iOS, `"height"` on Android).

```
import { AppKeyboardAvoidingView } from "@components/AppKeyboardAvoidingView";

<Modal ...>
  <AppKeyboardAvoidingView style={{ flex: 1 }}>
    { /* modal content with TextInput */ }
  </AppKeyboardAvoidingView>
</Modal>
```

Do **not** write `<KeyboardAvoidingView behavior={Platform.OS === "ios" ? "padding" : "height"}>` inline in new modals — use the wrapper so the behavior stays consistent and reviewable in one place.

System Architecture

The project uses a monorepo structure managed by pnpm workspaces. It is built with Node.js 24 and TypeScript 5.9.

Core Technologies:

- **Mobile App:** Expo / React Native
- **API Framework:** Express 5 (served at `/api`)
- **Database:** PostgreSQL with Drizzle ORM
- **Validation:** Zod
- **AI:** OpenAI via Replit AI Integrations

UI/UX and Features:

- **Dashboard:** Provides an overview of stats, recent cooks, and quick actions.
- **Grill Profiles:** Allows CRUD operations for managing different grills, including their history and statistics.
- **Cook Logger:** Records cook sessions with details like food type, weight, target temperatures, status, notes, and ratings.
- **AI Assistant & Predictions:** Offers natural language BBQ guidance and intelligent cook time predictions based on various parameters.
- **Temperature Monitoring:** Supports uploading data from various temperature probes (MEATER, ThermoWorks, Inkbird, Govee) and displays historical data with charts.
- **Recipes:** Enables browsing, searching, favoriting, and managing BBQ recipes.
- **Alerts:** Users can set temperature thresholds and rules for notifications.
- **Frozen Meat Planning:** Cooks can be flagged frozen with a chosen thaw method (refrigerator / cold-water / microwave / counter / cook-from-frozen). The Plan tab + AI predict route factor thaw + temper time into estimates; notifications fire when thaw and temper windows complete. The frozen-from flag is persisted on the Cook record so the Cook detail view shows the original state, not just the planning input.
- **Authentication:** Clerk (`@clerk/expo`). Email + password is primary. Google SSO works in dev and (once enabled in the prod Clerk instance) in production. Apple Sign-In is shown only when `AppleAuthentication.isAvailableAsync()` returns true; on iOS the native Apple flow exchanges an identity token via Clerk's `oauth_token_apple` strategy. All auth screens (`sign-in`, `sign-up`, `set-username`) render with the dark palette unconditionally via `useAuthColors()` regardless of system theme. Sign-in / sign-up handlers map Clerk error codes to user-friendly messages.
- **Account Deletion (Apple compliance):** Reachable from the More tab → "Delete account". `DELETE /api/profile/me` wipes the user's cooks, temperature readings, alerts, AI conversations, custom meat cuts, grills, MEATER/ThermoWorks credentials, subscription entitlements, and AI-analyze events in a single transaction, then deletes the Clerk user. If Clerk delete fails the data is still gone and the client signs the user out with a "contact support" message — no orphan-data state is possible.

Technical Implementations:

- **API Codegen:** Utilizes Orval for generating API hooks and Zod schemas from an OpenAPI specification.
- **Build System:** esbuild for CJS bundles.

- **Database Schema:** Key tables include `grills`, `cooks`, `recipes`, `temperature_readings`, `alerts`, and `conversations / messages`.
- **Environment Variables:** Server-side vars managed through `DATABASE_URL`, `AI_INTEGRATIONS_OPENAI_BASE_URL`, `AI_INTEGRATIONS_OPENAI_API_KEY`, and `SESSION_SECRET`. Mobile `EXPO_PUBLIC_*` vars are fully documented and audited in `artifacts/knowyourpit/ENV.md`.
- **Shared Brand Assets (lib/brand-assets):** Canonical package holding three files: `app-icon.png` (the circular grill icon), `logo.png` (the brand logo — distinct from the icon for future design flexibility), and `wordmark.png`. The marketing site (`public/icon.png` → `app-icon`, `public/logo.png` → `logo`, `public/wordmark.png`), `promo-video` (`public/brand/app-icon.png`, `app-logo.png` → `logo`, `marketing-logo.png` → `logo`, `wordmark.png`), and mobile app (`assets/images/icon.png` → `app-icon`, `assets/images/logo.png` → `logo`) all point to these files via relative symlinks — updating a file in one place propagates everywhere.
- **Marketing Website (artifacts/marketing):** A React + Vite application serving the landing page, privacy policy, terms of service, and support pages. It includes a contact form backend integrated with the API server.
- **Production Routing:** The deployed project hosts both the API server and the marketing static site. `/api/*` and `/health` are handled by the API server, while all other paths are served by the marketing site. Custom domain setup involves specific DNS records for `knowyourpit.com` and `www.knowyourpit.com`.

External Dependencies

- **OpenAI:** Integrated for AI assistant and prediction functionalities through Replit AI Integrations (gpt-5.2).
- **PostgreSQL:** Used as the primary database, managed with Drizzle ORM.
- **Expo/React Native:** Framework for mobile application development.
- **MEATER, ThermoWorks, Inkbird, Govee:** External temperature probe brands supported for data upload.
- **Apple Push Notification Service (APNS):** Implicitly used for mobile alerts and notifications via Expo.
- **Clerk:** Potentially used for authentication, though the contact form is not behind Clerk auth.
- **express-rate-limit:** Middleware used on the API server for rate limiting, specifically on the contact form endpoint.
- **Let's Encrypt:** Used for automatic TLS certificate provisioning for custom domains.
- **RevenueCat:** Subscription payroll — `pro` entitlement (monthly + annual). Mobile uses `react-native-purchases`; admin scripts in `scripts/src/{seedRevenueCatProducts,grantPro,revokePro}.ts` use `@replit/revenuecat-sdk` via the Replit RevenueCat connector. Server-side gates live in `artifacts/api-server/src/lib/paywall.ts` and emit a uniform 402 response via `respondPaywall(res, ...)`. Set `PAYWALL_ENABLED=false` to bypass every gate globally.
- **App Store Connect API Key (EAS submit):** Used by `artifacts/knowyourpit/scripts/submit-ios.sh` to upload iOS builds to TestFlight.

Current active key: ID `3WTDG9D596`, Issuer ID `2548969f-a92c-4ab7-b550-342a8afa0b37` (rotated from `3J5AF7DP8R` on 2026-05-09 — old key was revoked in App Store Connect). Key content stored in the `ASC_API_KEY_P8` Replit secret. Key ID is also set in `ASC_API_KEY_ID` secret and hardcoded in `artifacts/knowyourpit/eas.json` under `submit.production.ios.ascApiKeyId`.

Database Schema Sync Behaviour

`drizzle-kit push --force` only creates and alters tables that exist in the Drizzle schema; it does **not** drop tables that have been removed from the schema. To prevent orphan tables accumulating in the dev database after schema removals, the post-merge setup script (`scripts/post-merge.sh`) runs two steps in order:

1. **drop-orphans** (`lib/db/src/drop-orphan-tables.ts`) — queries `pg_tables` for every table in the `public` schema, compares against the Drizzle schema exports, and issues `DROP TABLE ... CASCADE` for any table that is absent from the schema.
2. **push-force** — pushes the current schema to the dev database, auto-approving any destructive column/index changes.

This means removing a table from `lib/db/src/schema/index.ts` and merging the task is sufficient to drop it from dev on the next post-merge run. No manual `DROP TABLE` step is needed.

Safety gate: `drop-orphan-tables.ts` requires `ALLOW_ORPHAN_DROP=true` to be set in the environment; otherwise it exits with an error. This prevents the script from running accidentally against a production database. The post-merge script sets this variable inline (`ALLOW_ORPHAN_DROP=true pnpm ...`). **Never** set `ALLOW_ORPHAN_DROP=true` when `DATABASE_URL` points to the production database.

Production orphan-table cleanup (runbook)

When a table is removed from the Drizzle schema and merged, it is automatically dropped from the **dev** database by the post-merge script. Production requires a manual step using the dedicated admin script:

```
# Step 1 – dry-run: inspect what will be dropped (no changes made)
DATABASE_URL=<prod-url> ALLOW_PROD_DROPS=1 \
  pnpm --filter @workspace/scripts run db:prod-drop-orphans

# Step 2 – live drop: actually remove the orphan tables
DATABASE_URL=<prod-url> ALLOW_PROD_DROPS=1 \
  pnpm --filter @workspace/scripts run db:prod-drop-orphans -- --confirm
```

Script: `scripts/src/drop-orphan-tables-prod.ts`

Safety gates (both must be satisfied before any table is dropped):

1. `ALLOW_PROD_DROPS=1` must be set — explicit acknowledgement that the target is production and drops are intentional.
2. `--confirm` must be passed as a CLI argument — without it the script runs in dry-run mode, printing the orphan list and exiting cleanly.

Always run the dry-run first to review the list before passing `--confirm`. The `drop-orphans` script (dev) is **never** run against production.

Ops Log

- 2026-05-27 — Build #105 / v1.0.10 submitted to TestFlight:** EAS build id `ee0a2484-79ef-4b8d-ac18-66179cb261ee`. Submission id `34653ddc-8aef-45ca-8fe3-45a010ab5e4a`. buildNumber 104→105, version 1.0.9→1.0.10. Features: check-in checkpoint rows + preview sheets in all timeline views (#974–#997); competition mode fully removed; PitMaster Score planSampleFactor fix (single-outlier grade fix); BLE diagnostics screen, Inkbird model expansion, iOS/Android permissions hardening (#998); live Inkbird channel temps in device cards (#1001); remember BLE permission denial across app restarts (#1003). Fix: useSmokerProfile baseUrl now uses EXPO_PUBLIC_API_URL first (was silently using empty string in production builds). No DB schema changes required.
- 2026-05-26 — Build #104 / v1.0.9 queued to TestFlight:** EAS build id `a78a5b5d-319d-4316-984e-55f3885d14a5`. buildNumber 103→104, version 1.0.8→1.0.9. Features: onboarding simplified from 4-slide horizontal pager to 2-page pop-up modal — Welcome + Feedback (#962); check-in pill pulses when due within 5 minutes (#957); adaptive ETA detection now applies to AI temperature-range check-in suggestions (#954); "Tuned to your N cooks on this grill" fingerprint callout in AI results modal (#966), live cook screen (#967), planned cook timeline, and multi-cook session screen (#969); fingerprint callout survives a replan (#970). No DB schema changes required.
- 2026-05-24 — Build #100 / v1.0.5 queued to TestFlight:** buildNumber 99→100, version 1.0.4→1.0.5. Features: multi-cook Home tab shows all active cooks side-by-side (#819); edit actual thaw/meat-on times on a live cook via clock icon (#795); auto-log check-in at scheduled milestones when a probe is connected (#820); reduce probe polling from 15 s to 15 min (#822); save frozen cook plan from Cook Now mode (#827); condensed PitMaster decision section (#828); decision section resets on new analysis (#829); unified "Check In with PitMaster" sheet — merged Check-In, Quick Log, and Ask PitMaster into single bottom sheet (#832); decision rationale collapses on navigate away (#833); mopping support — mop_frequency/mop_liquid DB columns, "mop" cook event type, mop schedule in plans, mop countdown in live cook, mop chips on log cards (#837); fix Plan Accuracy grade for frozen cooks — uses meatOnAt not thaw-start as actual-start anchor (#838); active-cook duration row on frozen cook cards (#841); thaw time row on frozen cook cards (#842); visual thaw-vs-active-cook bar on frozen cook cards (#843); thaw time in AI cook analysis context + PitMaster prompt section (#844). DB schema pushed to prod required: mop_frequency/mop_liquid on cooks, "mop" in cook_event_type enum, is_automatic/probe_source on cook_checksins.
- 2026-05-22 — Build #99 / v1.0.4 submitted to TestFlight:** EAS build id `c051e755-b184-49f6-9765-428d5937ea1a`. Submission id `61559779-8c9f-4078-9628-52e528aee1b9`. buildNumber 98→99. Features: thaw-start countdown on frozen cook cards (#762 — "START THAW" badge chip, countdown to thaw window open); spritz schedule sub-rows on cook timeline (#763 — teal dots between Meat On and Wrap/Pull-off showing each

spritz time + live countdown); wrap finish technique displayed as italic sub-line beneath wrap step.

- 2026-05-22 — Build #98 / v1.0.4 submitted to TestFlight:** EAS build id `08d59c98-a58e-4427-ba6f-2bc51bc6126f`. Submission id `8ea1afb3-eb50-4ea9-8622-f0dce25bea24` — status FINISHED (confirmed via EAS GraphQL). Feature: 24-hour minimum floor for refrigerator thaw method (#760) + `frozenThawOverlapsGrill` plan warning when thaw window extends past preheat start. buildNumber 97→98. Subsequent retry submissions (318d76f9, d639e495, f9531de2, aa3b9d4b) all ERRORED — expected, Apple rejected duplicate binary uploads after first submission succeeded.
- 2026-05-21 — Build #97 / v1.0.4 queued to TestFlight:** EAS build id `aff35a29-27c0-4540-932f-eddbda382216`. Resubmit — no code changes since #96.
- 2026-05-21 — Build #96 / v1.0.4 queued to TestFlight:** EAS build id `94828d8b-8536-4917-8112-f1a7a810d92d`. Features: cook photo attachments (#710, object storage, CookPhotosSection + AI photo analysis #719); cooking method chips in multi-cook (#707); remember last method per cut (#709, #711, #712, #720); per-cut technique memory for rub/spritz/wrap/injection/meatStartTemp (#725, #728); performance stats by technique (#713); per-cook editing in multi-cook session (#715, #716); spritz reminders notifications (#732), next-spritz countdown (#733), haptic nudge (#736); carry spritz cadence into AI plan (#734); technique summary on cook detail (#737); technique chips on cook log cards (#739); spritz cadence in AI analysis (#740); spritz liquid preference + PitMaster memory (#741); "Add to planned cook" button on cook detail. Pre-build: ONBOARDING_ALWAYS_SHOW flipped false, photo permission strings updated for general attachments, versionCode 2→3. Fix: expo-image-manipulator downgraded 55.0.16→14.0.8 (SDK 54 compat — build #95 errored XCODE_BUILD_ERROR: FileSystemUtilities.isReadableFile).
- 2026-05-18 — Build #94 / v1.0.3 queued to TestFlight:** EAS build id `13328fba-323c-4974-8ce4-9d1533f26a40`. Version bumped 1.0.2 → 1.0.3 because Apple locked the 1.0.2 train (ITMS-90186 + ITMS-90062 — previously approved version cannot receive new builds). Features: first-launch onboarding walkthrough (#684 — 6 slides, brand-orange palette, Skip→last slide, AsyncStorage+Clerk completion persistence, account-age gate so existing users never see it); onboarding replay from More → Help (#695); BetaWelcomeModal deleted (#696); Contact support row opens in-app support modal (#697, #700 — posts to `/api/contact` with source field); close button + back button in replay mode (#698, #701). Note: EAS_NO_VCS=1 required. Build #93 (v1.0.2) was discarded — never reached TestFlight.
- 2026-05-10/11 — Builds #80–#89 (resolved issues, condensed):**
 - Apple Sign-In** (builds #80–#82): Fixed "you are not authorized" errors. Correct pattern: call `signIn.create({ strategy: "oauth_token_apple", token, nonce })` first, then if `firstFactorVerification.status === "transferable"` call `signUp.create({ transfer: true })`. The `nonce` (from `Crypto.randomUUID()`) is required — Clerk's prod backend validates it for transfer-mode sign-up. New Apple users are routed to `/(auth)/set-username` after sign-up.
 - IAP paywall** (builds #86–#88 + server-side fix): Both subscriptions (`com.knowyourpit.pro.monthly` id=6764196256, `com.knowyourpit.pro.annual`

id=6764194128) resolved from MISSING_METADATA → READY_TO_SUBMIT after uploading review screenshots via ASC API. SubscriptionContext safety timer extended to 25 s. RC iOS SDK key: `appl_mhkVJhxDzwRmZTeUZSVShSwaimi`. Prices: \$4.99/mo, \$29.99/yr.

- **Email verification** (build #89): Sign-up now handles `unverifiedFields: ["email_address"]` — shows OTP screen, calls `prepareEmailAddressVerification` + `attemptEmailAddressVerification` before proceeding.
 - **App Store Review**: Pending — primary category + age rating must be set in ASC web UI before submitting. App version v1.0 build #88 is in PREPARE_FOR_SUBMISSION.
-

Database Schema

Extracted from Drizzle ORM schema files in `lib/db/src/schema/`. Each table shows SQL column names, types, and constraints.

Table: `grills`

Column	Type	Constraints
<code>id</code>	serial (int, PK auto)	PK
<code>user_id</code>	text	
<code>name</code>	text	NOT NULL
<code>type</code>	text	NOT NULL
<code>fuel_type</code>	text	
<code>brand</code>	text	
<code>model</code>	text	
<code>description</code>	text	
<code>cooking_surface_sq_in</code>	real (float)	
<code>min_temp_f</code>	real (float)	
<code>max_temp_f</code>	real (float)	
<code>num_probes</code>	integer	
<code>heat_zones</code>	integer	
<code>wifi_enabled</code>	boolean	default
<code>hopper_size_lbs</code>	real (float)	
<code>temp_range</code>	text	
<code>features</code>	text	
<code>notes</code>	text	
<code>image_url</code>	text	
<code>total_cooks</code>	integer	NOT NULL, default
<code>created_at</code>	timestamp (tz)	NOT NULL, default
<code>updated_at</code>	timestamp (tz)	NOT NULL, default

Table: cooks

Column	Type	Constraints
id	serial (int, PK auto)	PK
user_id	text	
grill_id	integer	
food_type	text	NOT NULL
weight_lbs	real (float)	
target_temp_f	real (float)	
cook_temp_f	real (float)	
status	text	NOT NULL, default
planned_start_at	timestamp (tz)	
actual_start_at	timestamp (tz)	
planned_end_at	timestamp (tz)	
actual_end_at	timestamp (tz)	
notes	text	
preheat_minutes	integer	
wrap_at_minutes	integer	
wrap_method	text	
wrap_temp_f	integer	
wrap_reason	text	
rest_minutes	integer	
rating_tenderness	integer	
rating_bark	integer	
rating_flavor	integer	
rating	integer	
session_id	text	
session_label	text	
session_notes	text	
sequence_data	jsonb	
confirmed_steps	jsonb	
analysis_result	jsonb	
analysis_history	jsonb	

Column	Type	Constraints
<code>from_frozen</code>	boolean	NOT NULL, default
<code>thaw_method</code>	text	
<code>actual_thaw_start_at</code>	timestamp (tz)	
<code>cooking_method</code>	text	
<code>injection</code>	text	
<code>spritz_frequency</code>	text	
<code>wrap_finish</code>	text	
<code>finish_time_range_lower</code>	timestamp (tz)	
<code>finish_time_range_upper</code>	timestamp (tz)	
<code>health_score</code>	text	
<code>health_score_reason</code>	text	
<code>created_at</code>	timestamp (tz)	NOT NULL, default
<code>updated_at</code>	timestamp (tz)	NOT NULL, default

Table: `temperature_readings`

Column	Type	Constraints
<code>id</code>	serial (int, PK auto)	PK
<code>cook_id</code>	integer	NOT NULL
<code>grill_id</code>	integer	
<code>probe_number</code>	integer	NOT NULL, default
<code>probe_name</code>	text	
<code>temp_f</code>	real (float)	NOT NULL
<code>recorded_at</code>	timestamp (tz)	NOT NULL
<code>source</code>	text	NOT NULL, default

Table: `alerts`

Column	Type	Constraints
<code>id</code>	serial (int, PK auto)	PK
<code>user_id</code>	text	
<code>cook_id</code>	integer	

Column	Type	Constraints
probe_number	integer	
alert_type	text	NOT NULL
threshold_temp_f	real (float)	NOT NULL
message	text	NOT NULL
is_active	boolean	NOT NULL, default
triggered_at	timestamp (tz)	
scheduled_notification_id	text	
created_at	timestamp (tz)	NOT NULL, default

Table: conversations

Column	Type	Constraints
id	serial (int, PK auto)	PK
user_id	text	
title	text	NOT NULL
created_at	timestamp (tz)	NOT NULL, default
updated_at	timestamp (tz)	NOT NULL, default

Table: messages

Column	Type	Constraints
id	serial (int, PK auto)	PK
conversation_id	integer	
role	text	NOT NULL
content	text	NOT NULL
created_at	timestamp (tz)	NOT NULL, default

Table: meater_credentials

Column	Type	Constraints
user_id	text	PK
access_token	text	NOT NULL

Column	Type	Constraints
token_expires_at	timestamp (tz)	NOT NULL

Table: thermoworks_credentials

Column	Type	Constraints
user_id	text	PK
email	text	NOT NULL
thermoworks_user_id	text	NOT NULL
thermoworks_account_id	text	NOT NULL
project_id	text	NOT NULL
id_token	text	NOT NULL
refresh_token	text	NOT NULL
token_expires_at	timestamp (tz)	NOT NULL

Table: contact_messages

Column	Type	Constraints
id	serial (int, PK auto)	PK
name	text	NOT NULL
email	text	NOT NULL
subject	text	NOT NULL
message	text	NOT NULL
source	text	NOT NULL, default
ip_address	varchar	
user_agent	text	
created_at	timestamp (tz)	NOT NULL, default

Table: ai_analyze_events

Column	Type	Constraints
id	serial (int, PK auto)	PK
user_id	text	NOT NULL

Column	Type	Constraints
<code>created_at</code>	timestamp (tz)	NOT NULL, default

Table: `frozen_timeline_events`

Column	Type	Constraints
<code>id</code>	serial (int, PK auto)	PK
<code>user_id</code>	text	NOT NULL
<code>created_at</code>	timestamp (tz)	NOT NULL, default

Table: `subscription_entitlements`

Column	Type	Constraints
<code>user_id</code>	text	PK
<code>is_pro</code>	boolean	NOT NULL, default
<code>expires_at</code>	timestamp (tz)	
<code>last_event_type</code>	text	NOT NULL
<code>last_event_at_ms</code>	bigint	
<code>updated_at</code>	timestamp (tz)	NOT NULL, default

Table: `custom_meat_cuts`

Column	Type	Constraints
<code>id</code>	serial (int, PK auto)	PK
<code>user_id</code>	text	NOT NULL
<code>name</code>	text	NOT NULL
<code>category</code>	text	NOT NULL
<code>target_temp_f</code>	real (float)	NOT NULL
<code>cook_temp_f</code>	real (float)	NOT NULL
<code>mins_per_lb</code>	real (float)	NOT NULL
<code>rest_mins</code>	integer	NOT NULL, default
<code>cook_method</code>	text	
<code>notes</code>	text	

Column	Type	Constraints
<code>created_at</code>	timestamp (tz)	NOT NULL, default
<code>updated_at</code>	timestamp (tz)	NOT NULL, default

Table: `live_activities`

Column	Type	Constraints
<code>activity_id</code>	text	PK
<code>user_id</code>	text	NOT NULL
<code>cook_id</code>	integer	NOT NULL
<code>push_token</code>	text	NOT NULL
<code>created_at</code>	timestamp (tz)	NOT NULL, default
<code>updated_at</code>	timestamp (tz)	

Table: `cook_checkins`

Column	Type	Constraints
<code>id</code>	serial (int, PK auto)	PK
<code>cook_id</code>	integer	NOT NULL
<code>scheduled_at</code>	timestamp (tz)	NOT NULL
<code>fired_at</code>	timestamp (tz)	
<code>internal_temp_f</code>	real (float)	
<code>pit_temp_f</code>	real (float)	
<code>status_flag</code>	enum	
<code>user_note</code>	text	
<code>photo_key</code>	text	
<code>ai_guidance_shown</code>	text	
<code>auto_dismissed</code>	boolean	NOT NULL, default
<code>is_automatic</code>	boolean	NOT NULL, default
<code>probe_source</code>	text	
<code>phase_label</code>	text	
<code>phase_key</code>	text	
<code>created_at</code>	timestamp (tz)	NOT NULL, default

Column	Type	Constraints
updated_at	timestamp (tz)	NOT NULL, default

Table: cook_events

Column	Type	Constraints
id	serial (int, PK auto)	PK
cook_id	integer	NOT NULL
occurred_at	timestamp (tz)	NOT NULL, default
event_type	enum	NOT NULL
note	text	
metadata	jsonb	
created_at	timestamp (tz)	NOT NULL, default

Table: cook_photos

Column	Type	Constraints
id	serial (int, PK auto)	PK
cook_id	integer	NOT NULL
user_id	text	NOT NULL
storage_key	text	NOT NULL
taken_at	timestamp (tz)	
created_at	timestamp (tz)	NOT NULL, default
pending_delete	boolean	NOT NULL, default

API Reference

All routes are prefixed with `/api`. Authenticated unless otherwise noted. Request/response fields shown for the primary content type; `?` suffix means nullable.

Ai

POST `/ai/chat`

Send a message to the AI assistant

Request body: AiChatBody

Field	Type	Required	Notes
<code>message</code>	string	✓	
<code>context</code>	string?		Optional JSON context (active cook, selected grill, etc.)

Response 200: AiChatResponse

Field	Type	Required	Notes
<code>reply</code>	string	✓	
<code>suggestions</code>	string[]	✓	

POST `/ai/predict`

Predict cook start and finish times based on grill and food history

Request body: AiPredictBody

Field	Type	Required	Notes
<code>grillId</code>	integer?		
<code>foodType</code>	string	✓	
<code>weightLbs</code>	number?		
<code>cookTempF</code>	number?		
<code>targetTempF</code>	number?		
<code>desiredFinishAt</code>	string?		
<code>preheatMinutes</code>	integer?		Minutes needed to start and bring the grill up to cook temperature
<code>outdoorTempF</code>	number?		Outdoor ambient temperature in Fahrenheit, used to adjust cook time

Field	Type	Required	Notes
			estimates. May be either current conditions or a forecast for the cook day (see <code>outdoorTempIsForecast</code>).
<code>outdoorTempIsForecast</code>	boolean?		True when <code>outdoorTempF</code> is a forecasted value for the cook day (Pro). False or omitted when it is the current ambient temperature.
<code>fromFrozen</code>	boolean?		True when the meat is starting from a fully frozen state and needs to be thawed before cooking. PitMaster will factor in thaw timing, surface drying, and dry-brine adjustments.
<code>thawMethod</code>	string?		Chosen thaw method when <code>fromFrozen</code> is true. "fridge" is slow and safest; "cold_water" is faster but requires more attention. Used to tailor PitMaster's thaw and surface-dry guidance.
<code>cookingMethod</code>	string?		Cooking technique chosen by the user (e.g. "Low & Slow", "Hot & Fast", "Rotisserie", "Reverse Sear"). Influences time estimate and wrap guidance.
<code>injection</code>	string?		Whether the meat is injected ("Injected" or "Not Injected"). Injected cuts retain more moisture and experience a shorter stall.
<code>spritzFrequency</code>	string?		How often the user plans to spritz/mop the meat during the cook (e.g. "No Spritz", "Every 30 min", "Every Hour", "Once at Stall"). Affects bark development and moisture retention.
<code>wrapFinish</code>	string?		User's intended wrap or finish method (e.g. "No Wrap", "Butcher Paper at Stall", "Foil at Stall (Texas Crutch)", "Foil Boat"). PitMaster should align wrap recommendation with this preference.
<code>meatStartTemp</code>	string?		Starting temperature state of the meat ("Cold from Fridge" or "Tempered to Room Temp"). Cold-from-fridge meat adds ~20-30 min to large cuts; tempered meat cooks at baseline pace.

Field	Type	Required	Notes
<code>notes</code>	string?		Optional free-text cook notes from the user (e.g. wood choice, rub details, special instructions). PitMaster factors these into timing, technique, and tip recommendations.

Response 200: AiPredictResponse

Field	Type	Required	Notes
<code>estimatedDurationMinutes</code>	number	✓	Active cook time only (food on → off grill), excluding preheat and rest
<code>preheatMinutes</code>	integer	✓	Minutes needed to start and bring the grill up to cook temperature
<code>grillLightAt</code>	string?	✓	When to light/start the grill, accounting for preheat time
<code>suggestedStartAt</code>	string?	✓	When to put food on the grill
<code>estimatedFinishAt</code>	string?	✓	When food comes off the grill (before rest)
<code>serveAt</code>	string?	✓	When food is ready to serve (after rest)
<code>wrap</code>	object	✓	
<code>confidence</code>	string	✓	enum: low, medium, high
<code>rationale</code>	string	✓	
<code>tips</code>	string[]	✓	
<code>checkins</code>	AiCheckinItem[]	✓	AI-generated check-in schedule tailored to this specific meat, technique, and cook plan. Each item fires at <code>offsetMinutes</code> after <code>meatOnAt</code> .
<code>fingerprintApplied</code>	boolean	✓	True when the per-grill fingerprint adjusted the time estimate or note
<code>fingerprintNote</code>	string?	✓	Human-readable note about how the grill's fingerprint was applied
<code>fingerprintSource</code>	string?	✓	Source of the fingerprint adjustment (per-grill learned pace, user-wide fallback, or pit bias only)

Field	Type	Required	Notes
<code>recommendedServeAt</code>	string?		When <code>fromFrozen</code> is true and the user's <code>desiredFinishAt</code> is too soon for a full thaw, PitMaster may suggest a later serve time that allows for proper thawing. Null when not applicable.
<code>recommendedServeReason</code>	string?		Human-readable explanation for the <code>recommendedServeAt</code> adjustment (e.g., why thawing requires more lead time).

POST `/ai/multi-cook`

Sequence multiple cooks to all be ready at the same serve time

Request body: MultiCookBody

Field	Type	Required	Notes
<code>items</code>	MultiCookItem[]	✓	
<code>serveAt</code>	string	✓	Target time when all food should be ready to serve.
<code>outdoorTempF</code>	number?		Outdoor ambient temperature in Fahrenheit. May be current conditions or a forecast for the cook day (see <code>outdoorTempsIsForecast</code>).
<code>outdoorTempIsForecast</code>	boolean?		True when <code>outdoorTempF</code> is a forecasted value for the cook day (Pro). False or omitted when it is the current ambient temperature.
<code>notes</code>	string?		Optional session-level free-text notes (e.g. wood choice, rub, special instructions). PitMaster factors these into timing and tips for all items in the session.

Response 200: MultiCookResponse

Field	Type	Required	Notes
<code>schedule</code>	MultiCookScheduleItem[]	✓	All cook items sorted by <code>grillLightAt</code> (earliest first)
<code>serveAt</code>	string	✓	

Field	Type	Required	Notes
<code>summary</code>	string	✓	One sentence summary of the overall sequencing plan

Alerts

GET /alerts

List temperature alerts

Response 200 (array of Alert)

Field	Type	Required	Notes
<code>id</code>	integer	✓	
<code>cookId</code>	integer?	✓	
<code>probeNumber</code>	integer?	✓	
<code>alertType</code>	string	✓	enum: min_temp, max_temp, target_reached, stall_detected, time_before_serve
<code>thresholdTempF</code>	number	✓	
<code>message</code>	string	✓	
<code>isActive</code>	boolean	✓	
<code>triggeredAt</code>	string?	✓	
<code>scheduledNotificationId</code>	string?		
<code>createdAt</code>	string	✓	

POST /alerts

Create a temperature alert

Request body: CreateAlertBody

Field	Type	Required	Notes
<code>cookId</code>	integer?		
<code>probeNumber</code>	integer?		
<code>alertType</code>	string	✓	enum: min_temp, max_temp, target_reached, stall_detected, time_before_serve
<code>thresholdTempF</code>	number	✓	
<code>message</code>	string	✓	

Field	Type	Required	Notes
scheduledNotificationId	string?		

Response 201: Alert

Field	Type	Required	Notes
id	integer	✓	
cookId	integer?	✓	
probeNumber	integer?	✓	
alertType	string	✓	enum: min_temp, max_temp, target_reached, stall_detected, time_before_serve
thresholdTempF	number	✓	
message	string	✓	
isActive	boolean	✓	
triggeredAt	string?	✓	
scheduledNotificationId	string?		
createdAt	string	✓	

PATCH /alerts/{id}**Update an alert (mark triggered or store scheduled notification ID)****Path params:** id (integer)**Request body: PatchAlertBody**

Field	Type	Required	Notes
triggered	boolean		
scheduledNotificationId	string?		

Response 200: Alert

Field	Type	Required	Notes
id	integer	✓	
cookId	integer?	✓	
probeNumber	integer?	✓	
alertType	string	✓	enum: min_temp, max_temp, target_reached, stall_detected, time_before_serve
thresholdTempF	number	✓	

Field	Type	Required	Notes
message	string	✓	
isActive	boolean	✓	
triggeredAt	string?	✓	
scheduledNotificationId	string?		
createdAt	string	✓	

DELETE /alerts/{id}

Delete an alert

Path params: id (integer)

Other responses: 204 — Deleted

Cooks

GET /cooks/technique-stats

Get per-technique performance stats

Returns per-cooking-technique aggregates (avg rating, cook count, top meat type) for techniques with at least 2 completed, rated cooks.

Response 200 (array of TechniqueStatsItem)

Field	Type	Required	Notes
technique	string	✓	Cooking technique name (e.g. Low & Slow, Hot & Fast)
cookCount	integer	✓	Number of completed, rated cooks using this technique
avgRating	number	✓	Average overall rating (1-5) across those cooks
topMeatType	string?	✓	Most frequently cooked meat type for this technique

GET /cooks

List all cook sessions

Query params: grillId, status

Response 200 (array of Cook)

Field	Type	Required	Notes
id	integer	✓	
grillId	integer?	✓	

Field	Type	Required	Notes
grillName	string?	✓	
foodType	string	✓	
weightLbs	number?	✓	
targetTempF	number?	✓	
cookTempF	number?	✓	
status	string	✓	enum: planned, active, completed, cancelled
plannedStartAt	string?	✓	
actualStartAt	string?	✓	
plannedEndAt	string?	✓	
actualEndAt	string?	✓	
notes	string?	✓	
preheatMinutes	integer?	✓	Minutes needed to start and bring the grill up to cook temperature
wrapAtMinutes	integer?	✓	Minutes into the cook when meat should be wrapped
wrapMethod	string?	✓	Wrapping method recommended
wrapTempF	integer?	✓	Internal meat temp at which to wrap
wrapReason	string?	✓	Explanation of why and how to wrap
restMinutes	integer?	✓	Recommended rest time after pulling from grill
ratingTenderness	integer?	✓	1-5 rating for meat tenderness
ratingBark	integer?	✓	1-5 rating for bark/crust/exterior color
ratingFlavor	integer?	✓	1-5 rating for overall flavor
rating	integer?	✓	Overall score (1-5), computed as average of sub-ratings
sessionId	string?	✓	UUID grouping cooks that were saved together from the Multi-Cook Sequencer
confirmedSteps	object?	✓	Map of step keys to ISO timestamps of when the user confirmed each step
fromFrozen	boolean	✓	True when this cook was planned starting from frozen meat (Frozen-to-Table mode)
thawMethod	string?	✓	Selected thaw method when fromFrozen is true

Field	Type	Required	Notes
<code>actualThawStartAt</code>	string?		The actual timestamp when the pitmaster moved the meat to begin thawing. Null until the pitmaster taps "Mark Thaw Started".
<code>cookingMethod</code>	string?		Cooking technique used (e.g. Low & Slow, Hot & Fast, Rotisserie)
<code>injection</code>	string?		Injection technique used (e.g. Not Injected, Injected)
<code>spritzFrequency</code>	string?		How often the cook was spritzed/mopped (e.g. Every Hour, No Spritz)
<code>wrapFinish</code>	string?		Wrap or finish method used (e.g. Butcher Paper at Stall, No Wrap)
<code>finishTimeRangeLower</code>	string?		Lower bound of the finish time confidence range
<code>finishTimeRangeUpper</code>	string?		Upper bound of the finish time confidence range
<code>healthScore</code>	string?		Cook health letter grade (A, B, C, D, F)
<code>healthScoreReason</code>	string?		One-line explanation of the health score
<code>currentTempF</code>	number?		Latest internal probe temperature reading for this cook (null when no readings exist)
<code>photoCount</code>	integer		Number of photos attached to this cook
<code>createdAt</code>	string	✓	
<code>updatedAt</code>	string	✓	

POST /cooks

Create a new cook session

Request body: CreateCookBody

Field	Type	Required	Notes
<code>grillId</code>	integer?		
<code>foodType</code>	string	✓	
<code>weightLbs</code>	number?		
<code>targetTempF</code>	number?		
<code>cookTempF</code>	number?		
<code>status</code>	string		enum: planned, active, completed, cancelled
<code>plannedStartAt</code>	string?		

Field	Type	Required	Notes
plannedEndAt	string?		
actualStartAt	string?		When the cook actually started (for logging past/completed cooks)
actualEndAt	string?		When the cook actually ended (for logging past/completed cooks)
notes	string?		
preheatMinutes	integer?		Minutes needed to start and bring the grill up to cook temperature
wrapAtMinutes	integer?		Minutes into the cook when meat should be wrapped
wrapMethod	string?		enum: foil, butcher_paper, none, null
wrapTempF	integer?		
wrapReason	string?		
restMinutes	integer?		Recommended rest time after pulling from grill
sessionId	string?		UUID grouping cooks that were saved together from the Multi-Cook Sequencer
recipeId	integer?		
fromFrozen	boolean?		True when planning the cook starting from frozen meat
thawMethod	string?		Selected thaw method when fromFrozen is true
cookingMethod	string?		Cooking technique used (e.g. Low & Slow, Hot & Fast, Rotisserie)
injection	string?		Injection technique used (e.g. Not Injected, Injected)
spritzFrequency	string?		How often the cook was spritzed/mopped (e.g. Every Hour, No Spritz)
wrapFinish	string?		Wrap or finish method used (e.g. Butcher Paper at Stall, No Wrap)

Response 201: Cook

Field	Type	Required	Notes
id	integer	✓	
grillId	integer?	✓	
grillName	string?	✓	
foodType	string	✓	

Field	Type	Required	Notes
weightLbs	number?	✓	
targetTempF	number?	✓	
cookTempF	number?	✓	
status	string	✓	enum: planned, active, completed, cancelled
plannedStartAt	string?	✓	
actualStartAt	string?	✓	
plannedEndAt	string?	✓	
actualEndAt	string?	✓	
notes	string?	✓	
preheatMinutes	integer?	✓	Minutes needed to start and bring the grill up to cook temperature
wrapAtMinutes	integer?	✓	Minutes into the cook when meat should be wrapped
wrapMethod	string?	✓	Wrapping method recommended
wrapTempF	integer?	✓	Internal meat temp at which to wrap
wrapReason	string?	✓	Explanation of why and how to wrap
restMinutes	integer?	✓	Recommended rest time after pulling from grill
ratingTenderness	integer?	✓	1-5 rating for meat tenderness
ratingBark	integer?	✓	1-5 rating for bark/crust/exterior color
ratingFlavor	integer?	✓	1-5 rating for overall flavor
rating	integer?	✓	Overall score (1-5), computed as average of sub-ratings
sessionId	string?	✓	UUID grouping cooks that were saved together from the Multi-Cook Sequencer
confirmedSteps	object?	✓	Map of step keys to ISO timestamps of when the user confirmed each step
fromFrozen	boolean	✓	True when this cook was planned starting from frozen meat (Frozen-to-Table mode)
thawMethod	string?	✓	Selected thaw method when fromFrozen is true
actualThawStartAt	string?		The actual timestamp when the pitmaster moved the meat to begin thawing. Null until the pitmaster taps "Mark Thaw Started".

Field	Type	Required	Notes
cookingMethod	string?		Cooking technique used (e.g. Low & Slow, Hot & Fast, Rotisserie)
injection	string?		Injection technique used (e.g. Not Injected, Injected)
spritzFrequency	string?		How often the cook was spritzed/mopped (e.g. Every Hour, No Spritz)
wrapFinish	string?		Wrap or finish method used (e.g. Butcher Paper at Stall, No Wrap)
finishTimeRangeLower	string?		Lower bound of the finish time confidence range
finishTimeRangeUpper	string?		Upper bound of the finish time confidence range
healthScore	string?		Cook health letter grade (A, B, C, D, F)
healthScoreReason	string?		One-line explanation of the health score
currentTempF	number?		Latest internal probe temperature reading for this cook (null when no readings exist)
photoCount	integer		Number of photos attached to this cook
createdAt	string	✓	
updatedAt	string	✓	

GET /cooks/{id}

Get a cook session by ID

Path params: `id` (integer)

Response 200: Cook

Field	Type	Required	Notes
id	integer	✓	
grillId	integer?	✓	
grillName	string?	✓	
foodType	string	✓	
weightLbs	number?	✓	
targetTempF	number?	✓	
cookTempF	number?	✓	
status	string	✓	enum: planned, active, completed, cancelled

Field	Type	Required	Notes
plannedStartAt	string?	✓	
actualStartAt	string?	✓	
plannedEndAt	string?	✓	
actualEndAt	string?	✓	
notes	string?	✓	
preheatMinutes	integer?	✓	Minutes needed to start and bring the grill up to cook temperature
wrapAtMinutes	integer?	✓	Minutes into the cook when meat should be wrapped
wrapMethod	string?	✓	Wrapping method recommended
wrapTempF	integer?	✓	Internal meat temp at which to wrap
wrapReason	string?	✓	Explanation of why and how to wrap
restMinutes	integer?	✓	Recommended rest time after pulling from grill
ratingTenderness	integer?	✓	1-5 rating for meat tenderness
ratingBark	integer?	✓	1-5 rating for bark/crust/exterior color
ratingFlavor	integer?	✓	1-5 rating for overall flavor
rating	integer?	✓	Overall score (1-5), computed as average of sub-ratings
sessionId	string?	✓	UUID grouping cooks that were saved together from the Multi-Cook Sequencer
confirmedSteps	object?	✓	Map of step keys to ISO timestamps of when the user confirmed each step
fromFrozen	boolean	✓	True when this cook was planned starting from frozen meat (Frozen-to-Table mode)
thawMethod	string?	✓	Selected thaw method when fromFrozen is true
actualThawStartAt	string?		The actual timestamp when the pitmaster moved the meat to begin thawing. Null until the pitmaster taps "Mark Thaw Started".
cookingMethod	string?		Cooking technique used (e.g. Low & Slow, Hot & Fast, Rotisserie)
injection	string?		Injection technique used (e.g. Not Injected, Injected)
spritzFrequency	string?		How often the cook was spritzed/mopped (e.g. Every Hour, No Spritz)

Field	Type	Required	Notes
wrapFinish	string?		Wrap or finish method used (e.g. Butcher Paper at Stall, No Wrap)
finishTimeRangeLower	string?		Lower bound of the finish time confidence range
finishTimeRangeUpper	string?		Upper bound of the finish time confidence range
healthScore	string?		Cook health letter grade (A, B, C, D, F)
healthScoreReason	string?		One-line explanation of the health score
currentTempF	number?		Latest internal probe temperature reading for this cook (null when no readings exist)
photoCount	integer		Number of photos attached to this cook
createdAt	string	✓	
updatedAt	string	✓	

Other responses: 404 — Not found

PATCH /cooks/{id}

Update a cook session

Path params: id (integer)

Request body: UpdateCookBody

Field	Type	Required	Notes
grillId	integer?		
foodType	string?		
weightLbs	number?		
targetTempF	number?		
cookTempF	number?		
status	string?		enum: planned, active, completed, cancelled, null
plannedStartAt	string?		
actualStartAt	string?		
plannedEndAt	string?		
actualEndAt	string?		
notes	string?		

Field	Type	Required	Notes
preheatMinutes	integer?		Minutes needed to start and bring the grill up to cook temperature
wrapAtMinutes	integer?		
wrapMethod	string?		enum: foil, butcher_paper, none, null
wrapTempF	integer?		
wrapReason	string?		
restMinutes	integer?		
ratingTenderness	integer?		
ratingBark	integer?		
ratingFlavor	integer?		
rating	integer?		
recipeId	integer?		
fromFrozen	boolean?		True when planning the cook starting from frozen meat
thawMethod	string?		Selected thaw method when fromFrozen is true
actualThawStartAt	string?		The actual timestamp when the pitmaster moved the meat to begin thawing. Set by tapping "Mark Thaw Started" on the banner.
confirmedSteps	object?		Map of step keys to ISO timestamps of when the user confirmed each step
cookingMethod	string?		Cooking technique used (e.g. Low & Slow, Hot & Fast, Rotisserie)
injection	string?		Injection technique used (e.g. Not Injected, Injected)
spritzFrequency	string?		How often the cook was spritzed/mopped (e.g. Every Hour, No Spritz)
wrapFinish	string?		Wrap or finish method used (e.g. Butcher Paper at Stall, No Wrap)

Response 200: Cook

Field	Type	Required	Notes
id	integer	✓	
grillId	integer?	✓	
grillName	string?	✓	

Field	Type	Required	Notes
foodType	string	✓	
weightLbs	number?	✓	
targetTempF	number?	✓	
cookTempF	number?	✓	
status	string	✓	enum: planned, active, completed, cancelled
plannedStartAt	string?	✓	
actualStartAt	string?	✓	
plannedEndAt	string?	✓	
actualEndAt	string?	✓	
notes	string?	✓	
preheatMinutes	integer?	✓	Minutes needed to start and bring the grill up to cook temperature
wrapAtMinutes	integer?	✓	Minutes into the cook when meat should be wrapped
wrapMethod	string?	✓	Wrapping method recommended
wrapTempF	integer?	✓	Internal meat temp at which to wrap
wrapReason	string?	✓	Explanation of why and how to wrap
restMinutes	integer?	✓	Recommended rest time after pulling from grill
ratingTenderness	integer?	✓	1-5 rating for meat tenderness
ratingBark	integer?	✓	1-5 rating for bark/crust/exterior color
ratingFlavor	integer?	✓	1-5 rating for overall flavor
rating	integer?	✓	Overall score (1-5), computed as average of sub-ratings
sessionId	string?	✓	UUID grouping cooks that were saved together from the Multi-Cook Sequencer
confirmedSteps	object?	✓	Map of step keys to ISO timestamps of when the user confirmed each step
fromFrozen	boolean	✓	True when this cook was planned starting from frozen meat (Frozen-to-Table mode)
thawMethod	string?	✓	Selected thaw method when fromFrozen is true
actualThawStartAt	string?		The actual timestamp when the pitmaster moved the meat to begin

Field	Type	Required	Notes
			thawing. Null until the pitmaster taps "Mark Thaw Started".
cookingMethod	string?		Cooking technique used (e.g. Low & Slow, Hot & Fast, Rotisserie)
injection	string?		Injection technique used (e.g. Not Injected, Injected)
spritzFrequency	string?		How often the cook was spritzed/mopped (e.g. Every Hour, No Spritz)
wrapFinish	string?		Wrap or finish method used (e.g. Butcher Paper at Stall, No Wrap)
finishTimeRangeLower	string?		Lower bound of the finish time confidence range
finishTimeRangeUpper	string?		Upper bound of the finish time confidence range
healthScore	string?		Cook health letter grade (A, B, C, D, F)
healthScoreReason	string?		One-line explanation of the health score
currentTempF	number?		Latest internal probe temperature reading for this cook (null when no readings exist)
photoCount	integer		Number of photos attached to this cook
createdAt	string	✓	
updatedAt	string	✓	

Other responses: 404 — Not found

DELETE /cooks/{id}

Delete a cook session

Path params: id (integer)

Other responses: 204 — Deleted

GET /cooks/{id}/checkins/schedule

Get the server-computed check-in schedule for a cook

Returns the AI-plan-anchored check-in schedule derived from the cook's sequence data. This is the single source of truth for check-in timing — the mobile app uses these times when scheduling local notifications.

Path params: id (integer)

Response 200 (array of CheckinScheduleItem)

Field	Type	Required	Notes
phaseKey	string	✓	
phaseLabel	string	✓	
scheduledAt	string	✓	
anchorType	string	✓	enum: sequence, percent

Other responses: 404 — Cook not found

GET /cooks/{id}/checkins

Get all check-ins for a cook

Path params: id (integer)

Response 200 (array of CookCheckin)

Field	Type	Required	Notes
id	integer	✓	
cookId	integer	✓	
scheduledAt	string	✓	
firedAt	string?	✓	
internalTempF	number?	✓	
pitTempF	number?	✓	
statusFlag	string	✓	enum: all_good, running_behind, flare_up, low_fuel
userNote	string?	✓	
photoKey	string?	✓	
aiGuidanceShown	string?	✓	
autoDismissed	boolean	✓	
isAutomatic	boolean	✓	
probeSource	string?	✓	
phaseLabel	string?	✓	
phaseKey	string?	✓	
createdAt	string	✓	
updatedAt	string	✓	

Other responses: 404 — Cook not found

POST /cooks/{id}/checkins

Record a check-in response for a cook**Path params:** `id` (integer)**Request body:** `CreateCookCheckinBody`

Field	Type	Required	Notes
<code>scheduledAt</code>	string	✓	
<code>internalTempF</code>	number?		
<code>pitTempF</code>	number?		
<code>statusFlag</code>	string?		enum: all_good, running_behind, flare_up, low_fuel, null
<code>userNote</code>	string?		
<code>photoKey</code>	string?		
<code>aiGuidanceShown</code>	string?		
<code>autoDismissed</code>	boolean		
<code>isAutomatic</code>	boolean		
<code>probeSource</code>	string?		
<code>phaseLabel</code>	string?		
<code>phaseKey</code>	string?		

Response 201: CookCheckin

Field	Type	Required	Notes
<code>id</code>	integer	✓	
<code>cookId</code>	integer	✓	
<code>scheduledAt</code>	string	✓	
<code>firedAt</code>	string?	✓	
<code>internalTempF</code>	number?	✓	
<code>pitTempF</code>	number?	✓	
<code>statusFlag</code>	string	✓	enum: all_good, running_behind, flare_up, low_fuel
<code>userNote</code>	string?	✓	
<code>photoKey</code>	string?	✓	
<code>aiGuidanceShown</code>	string?	✓	
<code>autoDismissed</code>	boolean	✓	
<code>isAutomatic</code>	boolean	✓	

Field	Type	Required	Notes
probeSource	string?	✓	
phaseLabel	string?	✓	
phaseKey	string?	✓	
createdAt	string	✓	
updatedAt	string	✓	

Other responses: 404 — Cook not found

GET /cooks/{id}/events

Get all quick-log events for a cook

Path params: id (integer)

Response 200 (array of CookLogEvent)

Field	Type	Required	Notes
id	integer	✓	
cookId	integer	✓	
occurredAt	string	✓	
eventType	string	✓	enum: lid_open, flare_up, spritz, mop, charcoal_add, wood_add, fuel_low, vent_adjust, user_note, proactive_alert, voice_note, ai_analysis
note	string?	✓	
metadata	object?	✓	
createdAt	string	✓	

Other responses: 404 — Cook not found

POST /cooks/{id}/events

Log a quick event during an active cook

Path params: id (integer)

Request body: CreateCookEventBody

Field	Type	Required	Notes
eventType	string	✓	enum: lid_open, flare_up, spritz, mop, charcoal_add, wood_add, fuel_low, vent_adjust, user_note, proactive_alert, voice_note
note	string?		
occurredAt	string?		

Field	Type	Required	Notes
metadata	object?		

Response 201: CookLogEvent

Field	Type	Required	Notes
id	integer	✓	
cookId	integer	✓	
occurredAt	string	✓	
eventType	string	✓	enum: lid_open, flare_up, spritz, mop, charcoal_add, wood_add, fuel_low, vent_adjust, user_note, proactive_alert, voice_note, ai_analysis
note	string?	✓	
metadata	object?	✓	
createdAt	string	✓	

Other responses: 404 — Cook not found**DELETE** /cooks/{id}/events/{eventId}**Retract (delete) a specific cook event****Path params:** id (integer), eventId (integer)**Other responses:** 204 — Event deleted | 404 — Cook or event not found**GET** /cooks/{id}/photos**List all photos attached to a cook****Path params:** id (integer)**Response 200 (array of CookPhoto)**

Field	Type	Required	Notes
id	integer	✓	
cookId	integer	✓	
userId	string	✓	
storageKey	string	✓	
signedUrl	string?	✓	Signed URL valid for 1 hour to fetch the photo
takenAt	string?	✓	
createdAt	string	✓	

Other responses: 404 — Cook not found

POST /cooks/{id}/photos

Upload a photo and attach it to a cook (multipart/form-data, max 10 MB, up to 10 photos per cook)

Path params: `id` (integer)

Request body (multipart/form-data): **CookPhotoUpload**

Field	Type	Required	Notes
<code>photo</code>	string	✓	Image file (JPEG, PNG, or WebP, max 10 MB)
<code>takenAt</code>	string		Optional timestamp when the photo was taken

Response 201: CookPhoto

Field	Type	Required	Notes
<code>id</code>	integer	✓	
<code>cookId</code>	integer	✓	
<code>userId</code>	string	✓	
<code>storageKey</code>	string	✓	
<code>signedUrl</code>	string?	✓	Signed URL valid for 1 hour to fetch the photo
<code>takenAt</code>	string?	✓	
<code>createdAt</code>	string	✓	

Other responses: 400 — Validation error (too many photos, bad file type, etc.) | 404 — Cook not found

DELETE /cooks/{id}/photos/{photoId}

Delete a photo from a cook

Path params: `id` (integer), `photoId` (integer)

Other responses: 204 — Photo deleted | 404 — Cook or photo not found

GET /cooks/{id}/health

Get the computed health score for a cook

Path params: `id` (integer)

Response 200: CookHealthScore

Field	Type	Required	Notes
<code>cookId</code>	integer	✓	
<code>grade</code>	string	✓	Letter grade: A, B, C, D, or F
<code>reason</code>	string	✓	One-line plain-English explanation of the grade

Field	Type	Required	Notes
factors	object	✓	Breakdown of individual score contributors
computedAt	string	✓	

Other responses: 404 — Cook not found

POST /cooks/{id}/live-activity

Register or update an iOS Live Activity push token for a cook

Path params: id (integer)

Request body: RegisterLiveActivityBody

Field	Type	Required	Notes
activityId	string	✓	
pushToken	string	✓	

Response 200: LiveActivityRegistration

Field	Type	Required	Notes
activityId	string	✓	
cookId	integer	✓	

Other responses: 404 — Cook not found

DELETE /cooks/{id}/live-activity

End all Live Activities tracked for a cook (server stops pushing)

Path params: id (integer)

Other responses: 204 — Activities ended

CustomMeatCuts

GET /custom-meat-cuts

List all custom meat cuts for the authenticated user

Response 200 (array of CustomMeatCut)

Field	Type	Required	Notes
id	integer	✓	
name	string	✓	
category	string	✓	

Field	Type	Required	Notes
targetTempF	number	✓	
cookTempF	number	✓	
minsPerLb	number	✓	
restMins	integer	✓	
cookMethod	string?	✓	
notes	string?	✓	
createdAt	string	✓	
updatedAt	string	✓	

POST /custom-meat-cuts

Create a custom meat cut

Request body: CreateCustomMeatCutBody

Field	Type	Required	Notes
name	string	✓	
category	string	✓	
targetTempF	number	✓	
cookTempF	number	✓	
minsPerLb	number	✓	
restMins	integer	✓	
cookMethod	string?		
notes	string?		

Response 201: CustomMeatCut

Field	Type	Required	Notes
id	integer	✓	
name	string	✓	
category	string	✓	
targetTempF	number	✓	
cookTempF	number	✓	
minsPerLb	number	✓	
restMins	integer	✓	
cookMethod	string?	✓	

Field	Type	Required	Notes
notes	string?	✓	
createdAt	string	✓	
updatedAt	string	✓	

PATCH /custom-meat-cuts/{id}

Update a custom meat cut

Path params: id (integer)

Request body: UpdateCustomMeatCutBody

Field	Type	Required	Notes
name	string?		
category	string?		
targetTempF	number?		
cookTempF	number?		
minsPerLb	number?		
restMins	integer?		
cookMethod	string?		
notes	string?		

Response 200: CustomMeatCut

Field	Type	Required	Notes
id	integer	✓	
name	string	✓	
category	string	✓	
targetTempF	number	✓	
cookTempF	number	✓	
minsPerLb	number	✓	
restMins	integer	✓	
cookMethod	string?	✓	
notes	string?	✓	
createdAt	string	✓	
updatedAt	string	✓	

Other responses: 404 — Not found

DELETE /custom-meat-cuts/{id}**Delete a custom meat cut****Path params:** id (integer)**Other responses:** 204 — Deleted

Dashboard

GET /dashboard/summary**Get dashboard summary stats****Response 200: DashboardSummary**

Field	Type	Required	Notes
totalCooks	integer	✓	
totalGrills	integer	✓	
activeCooks	integer	✓	
avgCookRating	number?	✓	
mostUsedGrill	string?	✓	
favoriteFood	string?	✓	
totalHoursCooking	number	✓	
activeAlerts	integer	✓	

GET /dashboard/recent-cooks**Get recent cook sessions****Response 200 (array of Cook)**

Field	Type	Required	Notes
id	integer	✓	
grillId	integer?	✓	
grillName	string?	✓	
foodType	string	✓	
weightLbs	number?	✓	
targetTempF	number?	✓	
cookTempF	number?	✓	
status	string	✓	enum: planned, active, completed, cancelled

Field	Type	Required	Notes
plannedStartAt	string?	✓	
actualStartAt	string?	✓	
plannedEndAt	string?	✓	
actualEndAt	string?	✓	
notes	string?	✓	
preheatMinutes	integer?	✓	Minutes needed to start and bring the grill up to cook temperature
wrapAtMinutes	integer?	✓	Minutes into the cook when meat should be wrapped
wrapMethod	string?	✓	Wrapping method recommended
wrapTempF	integer?	✓	Internal meat temp at which to wrap
wrapReason	string?	✓	Explanation of why and how to wrap
restMinutes	integer?	✓	Recommended rest time after pulling from grill
ratingTenderness	integer?	✓	1-5 rating for meat tenderness
ratingBark	integer?	✓	1-5 rating for bark/crust/exterior color
ratingFlavor	integer?	✓	1-5 rating for overall flavor
rating	integer?	✓	Overall score (1-5), computed as average of sub-ratings
sessionId	string?	✓	UUID grouping cooks that were saved together from the Multi-Cook Sequencer
confirmedSteps	object?	✓	Map of step keys to ISO timestamps of when the user confirmed each step
fromFrozen	boolean	✓	True when this cook was planned starting from frozen meat (Frozen-to-Table mode)
thawMethod	string?	✓	Selected thaw method when fromFrozen is true
actualThawStartAt	string?		The actual timestamp when the pitmaster moved the meat to begin thawing. Null until the pitmaster taps "Mark Thaw Started".
cookingMethod	string?		Cooking technique used (e.g. Low & Slow, Hot & Fast, Rotisserie)
injection	string?		Injection technique used (e.g. Not Injected, Injected)
spritzFrequency	string?		How often the cook was spritzed/mopped (e.g. Every Hour, No Spritz)

Field	Type	Required	Notes
wrapFinish	string?		Wrap or finish method used (e.g. Butcher Paper at Stall, No Wrap)
finishTimeRangeLower	string?		Lower bound of the finish time confidence range
finishTimeRangeUpper	string?		Upper bound of the finish time confidence range
healthScore	string?		Cook health letter grade (A, B, C, D, F)
healthScoreReason	string?		One-line explanation of the health score
currentTempF	number?		Latest internal probe temperature reading for this cook (null when no readings exist)
photoCount	integer		Number of photos attached to this cook
createdAt	string	✓	
updatedAt	string	✓	

GET /dashboard/temperature-history

Get aggregated temperature history across all cooks

Response 200 (array of TemperatureHistorySummary)

Field	Type	Required	Notes
cookId	integer	✓	
foodType	string	✓	
avgTempF	number	✓	
maxTempF	number	✓	
minTempF	number	✓	
readingCount	integer	✓	
cookedAt	string	✓	

Grills

GET /grills

List all grill profiles

Response 200 (array of Grill)

Field	Type	Required	Notes
id	integer	✓	
name	string	✓	
type	string	✓	charcoal, gas, pellet, electric, smoker, kamado, offset, etc.
fuelType	string?		charcoal, propane, natural gas, wood pellet, electric, wood, etc.
brand	string?	✓	
model	string?	✓	
description	string?	✓	
cookingSurfaceSqIn	number?	✓	
minTempF	number?	✓	
maxTempF	number?	✓	
numProbes	integer?	✓	
heatZones	integer?	✓	
wifiEnabled	boolean?	✓	
hopperSizeLbs	number?	✓	
tempRange	string?	✓	Human-readable temp range (e.g. '165°F - 500°F')
features	array?	✓	
notes	string?	✓	
imageUrl	string?	✓	
totalCooks	integer	✓	
cookCount	integer		Number of cooks logged on this grill (returned by list endpoint)
completedCookCount	integer		Number of cooks with status=completed on this grill
avgRating	number?		Average overall rating across rated cooks on this grill
lastCookAt	string?		Most recent actualStartAt of any completed cook on this grill
totalHours	number?		Total hours of completed cooks logged on this grill
mostCookedFood	string?		Food type most frequently cooked on this grill
createdAt	string	✓	

Field	Type	Required	Notes
updatedAt	string	✓	

POST /grills**Create a new grill profile****Request body: CreateGrillBody**

Field	Type	Required	Notes
name	string	✓	
type	string	✓	
fuelType	string?		
brand	string?		
model	string?		
description	string?		
cookingSurfaceSqIn	number?		
minTempF	number?		
maxTempF	number?		
numProbes	integer?		
heatZones	integer?		
wifiEnabled	boolean?		
hopperSizeLbs	number?		
tempRange	string?		
features	array?		
notes	string?		
imageUrl	string?		

Response 201: Grill

Field	Type	Required	Notes
id	integer	✓	
name	string	✓	
type	string	✓	charcoal, gas, pellet, electric, smoker, kamado, offset, etc.
fuelType	string?		charcoal, propane, natural gas, wood pellet, electric, wood, etc.
brand	string?	✓	

Field	Type	Required	Notes
model	string?	✓	
description	string?	✓	
cookingSurfaceSqIn	number?	✓	
minTempF	number?	✓	
maxTempF	number?	✓	
numProbes	integer?	✓	
heatZones	integer?	✓	
wifiEnabled	boolean?	✓	
hopperSizeLbs	number?	✓	
tempRange	string?	✓	Human-readable temp range (e.g. '165°F - 500°F')
features	array?	✓	
notes	string?	✓	
imageUrl	string?	✓	
totalCooks	integer	✓	
cookCount	integer		Number of cooks logged on this grill (returned by list endpoint)
completedCookCount	integer		Number of cooks with status=completed on this grill
avgRating	number?		Average overall rating across rated cooks on this grill
lastCookAt	string?		Most recent actualStartAt of any completed cook on this grill
totalHours	number?		Total hours of completed cooks logged on this grill
mostCookedFood	string?		Food type most frequently cooked on this grill
createdAt	string	✓	
updatedAt	string	✓	

GET /grills/{id}

Get a grill profile by ID

Path params: id (integer)

Response 200: Grill

Field	Type	Required	Notes
id	integer	✓	
name	string	✓	
type	string	✓	charcoal, gas, pellet, electric, smoker, kamado, offset, etc.
fuelType	string?		charcoal, propane, natural gas, wood pellet, electric, wood, etc.
brand	string?	✓	
model	string?	✓	
description	string?	✓	
cookingSurfaceSqIn	number?	✓	
minTempF	number?	✓	
maxTempF	number?	✓	
numProbes	integer?	✓	
heatZones	integer?	✓	
wifiEnabled	boolean?	✓	
hopperSizeLbs	number?	✓	
tempRange	string?	✓	Human-readable temp range (e.g. '165°F - 500°F')
features	array?	✓	
notes	string?	✓	
imageUrl	string?	✓	
totalCooks	integer	✓	
cookCount	integer		Number of cooks logged on this grill (returned by list endpoint)
completedCookCount	integer		Number of cooks with status=completed on this grill
avgRating	number?		Average overall rating across rated cooks on this grill
lastCookAt	string?		Most recent actualStartAt of any completed cook on this grill
totalHours	number?		Total hours of completed cooks logged on this grill
mostCookedFood	string?		Food type most frequently cooked on this grill
createdAt	string	✓	

Field	Type	Required	Notes
updatedAt	string	✓	

Other responses: 404 — Not found

PATCH /grills/{id}

Update a grill profile

Path params: id (integer)

Request body: UpdateGrillBody

Field	Type	Required	Notes
name	string?		
type	string?		
fuelType	string?		
brand	string?		
model	string?		
description	string?		
cookingSurfaceSqIn	number?		
minTempF	number?		
maxTempF	number?		
numProbes	integer?		
heatZones	integer?		
wifiEnabled	boolean?		
hopperSizeLbs	number?		
tempRange	string?		
features	array?		
notes	string?		
imageUrl	string?		

Response 200: Grill

Field	Type	Required	Notes
id	integer	✓	
name	string	✓	
type	string	✓	charcoal, gas, pellet, electric, smoker, kamado, offset, etc.

Field	Type	Required	Notes
<code>fuelType</code>	string?		charcoal, propane, natural gas, wood pellet, electric, wood, etc.
<code>brand</code>	string?	✓	
<code>model</code>	string?	✓	
<code>description</code>	string?	✓	
<code>cookingSurfaceSqIn</code>	number?	✓	
<code>minTempF</code>	number?	✓	
<code>maxTempF</code>	number?	✓	
<code>numProbes</code>	integer?	✓	
<code>heatZones</code>	integer?	✓	
<code>wifiEnabled</code>	boolean?	✓	
<code>hopperSizeLbs</code>	number?	✓	
<code>tempRange</code>	string?	✓	Human-readable temp range (e.g. '165°F - 500°F')
<code>features</code>	array?	✓	
<code>notes</code>	string?	✓	
<code>imageUrl</code>	string?	✓	
<code>totalCooks</code>	integer	✓	
<code>cookCount</code>	integer		Number of cooks logged on this grill (returned by list endpoint)
<code>completedCookCount</code>	integer		Number of cooks with status=completed on this grill
<code>avgRating</code>	number?		Average overall rating across rated cooks on this grill
<code>lastCookAt</code>	string?		Most recent actualStartAt of any completed cook on this grill
<code>totalHours</code>	number?		Total hours of completed cooks logged on this grill
<code>mostCookedFood</code>	string?		Food type most frequently cooked on this grill
<code>createdAt</code>	string	✓	
<code>updatedAt</code>	string	✓	

Other responses: 404 — Not found

DELETE `/grills/{id}`

Delete a grill profile**Path params:** `id` (integer)**Other responses:** 204 — Deleted`GET /grills/{id}/stats`**Get cook stats for a specific grill****Path params:** `id` (integer)**Response 200: GrillStats**

Field	Type	Required	Notes
<code>grillId</code>	integer	✓	
<code>totalCooks</code>	integer	✓	
<code>totalHours</code>	number	✓	
<code>avgCookDurationMinutes</code>	number	✓	
<code>mostCookedFood</code>	string?	✓	
<code>avgTargetTempF</code>	number?	✓	
<code>avgPitTempF</code>	number?	✓	Average achieved pit/ambient temperature across all readings
<code>pitTempVarianceF</code>	number?	✓	Average per-cook spread between max and min pit readings
<code>probeHighTempF</code>	number?	✓	Highest probe (meat) temp recorded on this grill
<code>totalReadings</code>	integer	✓	Total temperature readings logged for this grill

`GET /grills/{id}/fingerprint`**Get the learned per-grill calibration profile (fingerprint)****Path params:** `id` (integer)**Response 200: GrillFingerprint**

Field	Type	Required	Notes
<code>grillId</code>	integer	✓	
<code>cookCount</code>	integer	✓	Number of completed cooks on this grill
<code>confidenceLevel</code>	string	✓	enum: none, building, developing, established
<code>pitBiasF</code>	number?	✓	Average pit temperature bias vs set point (positive = runs hot)

Field	Type	Required	Notes
overshootF	number?	✓	Average pull temperature overshoot vs target (positive = overshoots)
durationByMeat	object	✓	
runLong	boolean?	✓	
runShort	boolean?	✓	

Other responses: 404 — Grill not found

GET /grills/{id}/insights

Get the learned per-grill calibration profile (available to all authenticated users)

Path params: id (integer)

Response 200: GrillInsights

Field	Type	Required	Notes
grillId	integer	✓	
cookCount	integer	✓	Number of completed cooks on this grill
confidenceLevel	string	✓	enum: none, building, developing, established
pitBiasF	number?	✓	Average pit temperature bias vs set point (positive = runs hot)
overshootF	number?	✓	Average pull temperature overshoot vs target (positive = overshoots)
durationByMeat	object	✓	

Other responses: 404 — Grill not found

GET /grills/{id}/temperature-history

Get recent temperature readings grouped by cook for a specific grill

Path params: id (integer)

Response 200: GrillTemperatureHistory

Field	Type	Required	Notes
grillId	integer	✓	
cooks	GrillTempCookEntry[]	✓	

Health

GET /healthz

Health check

Response 200: HealthStatus

Field	Type	Required	Notes
status	string	✓	

Meater

POST /meater/link

Link MEATER Cloud account

Request body: MeaterLinkBody

Field	Type	Required	Notes
email	string	✓	
password	string	✓	

Response 200: MeaterLinkedResponse

Field	Type	Required	Notes
linked	boolean	✓	

Other responses: 401 — Invalid MEATER credentials

DELETE /meater/unlink

Unlink MEATER Cloud account

Response 200: MeaterLinkedResponse

Field	Type	Required	Notes
linked	boolean	✓	

GET /meater/status

Get MEATER link status and device list

Response 200: MeaterStatusResponse

Field	Type	Required	Notes
linked	boolean	✓	
devices	MeaterDevice[]	✓	
tokenExpired	boolean		

GET /meater/readings

Get live readings from first active MEATER probe**Response 200: MeaterReadingsResponse**

Field	Type	Required	Notes
<code>linked</code>	boolean	✓	
<code>probes</code>	MeaterProbeReading[]	✓	
<code>tokenExpired</code>	boolean		

Temperature**POST** `/temperature/scan-image`**Scan an image for temperature readings using AI vision****Request body: TemperatureScanImageBody**

Field	Type	Required	Notes
<code>base64Image</code>	string	✓	Base64-encoded image data (JPG, PNG, or WEBP)
<code>mimeType</code>	string		MIME type of the image: image/jpeg, image/png, or image/webp

Response 200: TemperatureScanResult

Field	Type	Required	Notes
<code>readings</code>	ScannedReading[]	✓	
<code>cookDurationMinutes</code>	integer?		Total cook duration in minutes, or null if not determinable from the image
<code>noDataFound</code>	boolean	✓	True when the image had no readable temperature data
<code>rawExtraction</code>	string?		Raw text extracted from the image before structuring
<code>detectedFoodType</code>	string?		Meat or food type detected from app labels, cook log text, or graph annotations in the image
<code>detectedCookDate</code>	string?		Cook start date/time detected from the image (ISO 8601 UTC)

POST `/temperature/analyze-cook`**Analyze multiple cook images + notes to synthesize a cook timeline****Request body: AnalyzeCookBody**

Field	Type	Required	Notes
<code>images</code>	AnalyzeCookImageInput[]	✓	One or more thermometer/grill images from the cook (max 10)
<code>cookNotes</code>	string?		Optional free-text notes from the user about the cook
<code>cookId</code>	integer?		Optional cook ID — when provided, the analysis result is saved as a Pit Journal entry
<code>cookContext</code>	object?		Structured cook context for richer AI analysis

Response 200: AnalyzeCookResult

Field	Type	Required	Notes
<code>probes</code>	AnalyzedProbe[]	✓	Per-probe summary and synthesized time-series
<code>events</code>	CookEvent[]	✓	Detected cook events such as wrap, stall, spike
<code>cookDurationMinutes</code>	integer?		Total cook duration in minutes
<code>detectedFoodType</code>	string?		
<code>detectedCookDate</code>	string?		
<code>noDataFound</code>	boolean	✓	
<code>rawExtraction</code>	string?		

POST `/temperature/upload`**Upload temperature data from thermometer apps****Request body: TemperatureUploadBody**

Field	Type	Required	Notes
<code>cookId</code>	integer	✓	
<code>source</code>	string	✓	meater, thermoworks, inkbird, govee, csv, manual
<code>readings</code>	TemperatureReadingInput[]	✓	

Response 201: TemperatureUploadResult

Field	Type	Required	Notes
<code>inserted</code>	integer	✓	
<code>cookId</code>	integer	✓	

GET /temperature/readings**Get temperature readings for a cook session****Query params:** cookId**Response 200 (array of TemperatureReading)**

Field	Type	Required	Notes
id	integer	✓	
cookId	integer	✓	
probeNumber	integer	✓	
probeName	string?	✓	
tempF	number	✓	
recordedAt	string	✓	
source	string	✓	manual, meater, thermoworks, inkbird, govee, csv

Thermoworks

POST /thermoworks/link**Link ThermoWorks Cloud account****Request body: ThermoworksLinkBody**

Field	Type	Required	Notes
email	string	✓	
password	string	✓	

Response 200: ThermoworksLinkedResponse

Field	Type	Required	Notes
linked	boolean	✓	

Other responses: 401 — Invalid ThermoWorks credentials**DELETE** /thermoworks/unlink**Unlink ThermoWorks Cloud account****Response 200: ThermoworksLinkedResponse**

Field	Type	Required	Notes
linked	boolean	✓	

GET /thermoworks/status**Get ThermoWorks link status and device list****Response 200: ThermoworksStatusResponse**

Field	Type	Required	Notes
linked	boolean	✓	
devices	ThermoworksDevice[]	✓	
error	string		

GET /thermoworks/readings**Get live readings from active ThermoWorks probes****Response 200: ThermoworksReadingsResponse**

Field	Type	Required	Notes
linked	boolean	✓	
probes	ThermoworksProbeReading[]	✓	
error	string		

Mobile Screen Map

All screens in `artifacts/knowyourpit/app/`. Layout files are excluded.

Route	File	Description
<code>/[auth]/set-username</code>	<code>(auth)/set-username.tsx</code>	Authentication — choose username after OAuth sign-up
<code>/[auth]/sign-in</code>	<code>(auth)/sign-in.tsx</code>	Authentication — email/password and social sign-in
<code>/[auth]/sign-up</code>	<code>(auth)/sign-up.tsx</code>	Authentication — new account registration with email verification
<code>/[onboarding]/index</code>	<code>(onboarding)/index.tsx</code>	Home dashboard — active cooks overview and quick actions
<code>/[tabs]/ai</code>	<code>(tabs)/ai.tsx</code>	AI assistant — natural language BBQ guidance chat
<code>/[tabs]/cooks</code>	<code>(tabs)/cooks.tsx</code>	Cook history — list of all past and active cook sessions
<code>/[tabs]/index</code>	<code>(tabs)/index.tsx</code>	Home dashboard — active cooks overview and quick actions
<code>/[tabs]/more</code>	<code>(tabs)/more.tsx</code>	More tab — settings, profile, help, and account management
<code>/[tabs]/plan</code>	<code>(tabs)/plan.tsx</code>	Plan tab — frozen meat planning and cook time prediction
<code>/+not-found</code>	<code>+not-found.tsx</code>	404 screen — unmatched routes
<code>/alerts</code>	<code>alerts.tsx</code>	Temperature alerts — list, create, and manage cook alerts
<code>/ble-diagnostics</code>	<code>ble-diagnostics.tsx</code>	BLE diagnostics — Bluetooth device inspection and debug info
<code>/cooks/[id]</code>	<code>cooks/[id].tsx</code>	Cook detail — full cook record with timeline, check-ins, events, and photos
<code>/cooks/log</code>	<code>cooks/log.tsx</code>	Cook logger — create a new cook session with food type, weight, temps
<code>/devices</code>	<code>devices.tsx</code>	Devices — manage connected temperature probe devices (MEATER, Inkbird, etc.)

Route	File	Description
<code>/grills</code>	<code>grills.tsx</code>	Grill profiles — list, create, and manage grill configurations
<code>/index</code>	<code>index.tsx</code>	Home dashboard — active cooks overview and quick actions
<code>/pro-features</code>	<code>pro-features.tsx</code>	Pro features — RevenueCat paywall and subscription management
<code>/profile</code>	<code>profile.tsx</code>	User profile — account details, stats, and settings
<code>/sessions/[sessionId]</code>	<code>sessions/[sessionId].tsx</code>	Multi-cook session — view all cooks in a session grouped together
<code>/temperature</code>	<code>temperature.tsx</code>	Temperature — upload and browse historical temperature readings

API Server Routes

All TypeScript route modules under `artifacts/api-server/src/routes/` (recursive).

File	Resource / Purpose
<code>admin.ts</code>	Admin utilities and internal operations
<code>ai/bbqKnowledge.ts</code>	BBQ knowledge base lookup used by AI prompts
<code>ai/chat.ts</code>	AI chat — streaming and non-streaming message handling
<code>ai/index.ts</code>	AI router — mounts chat, predict, multi-cook sub-routes
<code>ai/insights.ts</code>	AI-generated cook insights and PitMaster analysis
<code>ai/meatBaselines.ts</code>	Meat baseline data and cook time reference tables
<code>ai/multiCook.ts</code>	Multi-cook sequencing — schedules parallel cooks for same serve time
<code>ai/predict.ts</code>	Cook time prediction using grill fingerprint + meat baselines
<code>ai/shared.ts</code>	Shared AI helpers (prompt builders, context assembly)
<code>alerts.ts</code>	Temperature alert CRUD — create, update, delete, trigger
<code>contact.ts</code>	Contact form submission (rate-limited, no auth required)
<code>conversations.ts</code>	AI conversation history — list, rename, delete
<code>cookCheckins.ts</code>	Cook check-in records, schedule, and auto-checkin support
<code>cookEvents.ts</code>	Quick-log cook events (spritz, mop, flare-up, fuel, etc.)
<code>cookPhotos.ts</code>	Cook photo upload and deletion (object storage, max 10 photos)
<code>cooks.ts</code>	Cook session CRUD, health scoring, and status management
<code>customMeatCuts.ts</code>	User-defined custom meat cut definitions
<code>dashboard.ts</code>	Dashboard summary, recent cooks, aggregated temperature history
<code>grills.ts</code>	Grill profile CRUD, fingerprint, insights, temperature history
<code>health.ts</code>	Health check endpoint (<code>/api/healthz</code>)
<code>index.ts</code>	Root router — mounts all sub-routers under <code>/api</code>

File	Resource / Purpose
<code>liveActivities.ts</code>	iOS Live Activity push token registration and teardown
<code>meater.ts</code>	MEATER Cloud integration — link, unlink, status, live readings
<code>paywall.ts</code>	RevenueCat webhook receiver and subscription entitlement sync
<code>profile.ts</code>	User profile fetch and full account deletion (Apple compliance)
<code>temperature/analyze.ts</code>	Analyze cook images + notes into a cook timeline
<code>temperature/analyzePrompt.ts</code>	AI prompt builder for temperature image analysis
<code>temperature/index.ts</code>	Temperature router — mounts scan, upload, readings sub-routes
<code>temperature/manual.ts</code>	Manual temperature reading upload and retrieval
<code>temperature/scan.ts</code>	AI vision scan of thermometer screenshots for readings
<code>temperature/shared.ts</code>	Shared temperature helpers (parsers, validators)
<code>thermoworks.ts</code>	ThermoWorks Cloud integration — link, unlink, status, live readings
<code>webhooks.ts</code>	Inbound webhook dispatcher (RevenueCat, future integrations)

Environment Variables

Mobile App Environment Variables

Canonical reference for every `EXPO_PUBLIC_*` variable consumed by the `artifacts/knowyourpit` package. Keep this file in sync whenever a variable is added, renamed, or removed — it is the single source of truth for the EAS dashboard audit.

How variables reach the app

Origin	Variables set
<code>eas.json</code> <code>build.production.env</code> (hardcoded)	<code>EXPO_PUBLIC_API_URL</code>
EAS secrets / <code>eas env</code> (must be configured manually)	<code>EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY_PROD</code> , <code>EXPO_PUBLIC_REVENUECAT_IOS_KEY</code> , <code>EXPO_PUBLIC_REVENUECAT_ANDROID_KEY</code>
Runtime injection by <code>scripts/build.js</code> (Replit web build)	<code>EXPO_PUBLIC_DOMAIN</code> , <code>EXPO_PUBLIC_REPL_ID</code> , <code>EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY</code> , <code>EXPO_PUBLIC_CLERK_PROXY_URL</code>
<code>package.json dev</code> script (Replit dev session)	<code>EXPO_PUBLIC_DOMAIN</code> , <code>EXPO_PUBLIC_REPL_ID</code> , <code>EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY</code>

Variables

`EXPO_PUBLIC_API_URL`

- **Purpose:** Base URL of the deployed API server.
- **Example:** `https://api.knowyourpit.com`
- **Set in:** `eas.json` `build.production.env` (hardcoded). Also set manually via `eas env:create` for `preview` and `development` profiles when testing against a real device.
- **Used in:** `app/_layout.tsx` , `hooks/usePaywallUsage.ts` , `hooks/useHomeInsights.ts` , `hooks/useMeaterReadings.ts` , `app/(tabs)/ai.tsx` , `app/(tabs)/plan.tsx` , `app/(tabs)/more.tsx` .
- **Fallback:** If unset, the app constructs the URL from `EXPO_PUBLIC_DOMAIN` .

`EXPO_PUBLIC_DOMAIN`

- **Purpose:** Replit deployment/dev domain (host only, no protocol). Used as a fallback API URL base when `EXPO_PUBLIC_API_URL` is not set.
- **Example:** `abc123.replit.app`

- **Set in:** Injected at runtime from `$REPLIT_DEV_DOMAIN` (dev) or `$REPLIT_INTERNAL_APP_DOMAIN` / `$REPLIT_DEV_DOMAIN` (build script). Never set in EAS; Replit-only.
 - **Used in:** `app/_layout.tsx`, `hooks/usePaywallUsage.ts`, `hooks/useHomeInsights.ts`, `hooks/useMeaterReadings.ts`, `hooks/useSmokerProfile.ts`, `app/(tabs)/ai.tsx`, `app/(tabs)/plan.tsx`, `app/(tabs)/more.tsx`, `scripts/build.js`.
-

EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY

- **Purpose:** Development Clerk publishable key (`pk_test_...`). Used in development and Replit web builds.
 - **Set in:** Injected at runtime from the `CLERK_PUBLISHABLE_KEY` Replit secret by `package.json dev` and `scripts/build.js`. Never stored directly in EAS.
 - **Used in:** `app/_layout.tsx`, `scripts/build.js`.
-

EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY_PROD

- **Purpose:** Production Clerk publishable key (`pk_live_...`). Takes priority over `EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY` in `app/_layout.tsx`.
 - **Set in:** Must be created as an EAS secret before every production build:


```
eas secret:create EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY_PROD pk_live_xxxx
```
 - **EAS environments:** `production`, `preview` (if testing against prod Clerk).
 - **Used in:** `app/_layout.tsx`.
-

EXPO_PUBLIC_CLERK_PROXY_URL

- **Purpose:** Full URL of the Clerk proxy (used in Replit dev to route Clerk requests through the Replit domain).
 - **Example:** `https://abc123.replit.app/api/clerk-proxy`
 - **Set in:** Injected at runtime by `scripts/build.js` from `process.env.CLERK_PROXY_URL`. Not set in EAS; Replit-only.
 - **Used in:** `app/_layout.tsx`.
-

EXPO_PUBLIC_REPL_ID

- **Purpose:** Replit REPL ID, used internally by `scripts/build.js` as a fallback for `process.env.REPL_ID`. Not consumed by any app screen or hook — only by the build script.
 - **Set in:** Injected at runtime from `$REPL_ID` by `package.json dev` and `scripts/build.js`. Not set in EAS.
 - **Used in:** `scripts/build.js`.
-

EXPO_PUBLIC_REVENUECAT_IOS_KEY

- **Purpose:** RevenueCat iOS public API key (`appl_...`).
- **Set in:** Must be configured in EAS for every environment that tests or ships in-app purchases:

```
eas env:create --scope project --name EXPO_PUBLIC_REVENUECAT_IOS_KEY \
  --value "appl_xxx" --environment development
eas env:create --scope project --name EXPO_PUBLIC_REVENUECAT_IOS_KEY \
  --value "appl_xxx" --environment preview
eas env:create --scope project --name EXPO_PUBLIC_REVENUECAT_IOS_KEY \
  --value "appl_xxx" --environment production
```

- **EAS environments:** `development`, `preview`, `production`.
- **Used in:** `contexts/SubscriptionContext.tsx`, `components/PaywallModal.tsx`.

`EXPO_PUBLIC_REVENUECAT_ANDROID_KEY`

- **Purpose:** RevenueCat Android public API key (`goog_...`).
- **Set in:** Same pattern as `EXPO_PUBLIC_REVENUECAT_IOS_KEY` above, for all EAS environments that build for Android.
- **EAS environments:** `development`, `preview`, `production`.
- **Used in:** `contexts/SubscriptionContext.tsx`, `components/PaywallModal.tsx`.

EAS environment matrix

Variable	development	preview	production	Source
<code>EXPO_PUBLIC_API_URL</code>	manual <code>eas env</code>	manual <code>eas env</code>	<code>eas.json</code> hardcoded	EAS
<code>EXPO_PUBLIC_DOMAIN</code>	—	—	—	Runtime (Replit only)
<code>EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY</code>	—	—	—	Runtime (Replit only)
<code>EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY_PROD</code>	—	optional	<code>eas secret</code>	EAS secret
<code>EXPO_PUBLIC_CLERK_PROXY_URL</code>	—	—	—	Runtime (Replit only)
<code>EXPO_PUBLIC_REPL_ID</code>	—	—	—	Runtime (Replit only)
<code>EXPO_PUBLIC_REVENUECAT_IOS_KEY</code>	<code>eas env</code> ✓	<code>eas env</code> ✓	<code>eas env</code> ✓	EAS

Variable	development	preview	production	Source
EXPO_PUBLIC_REVENUECAT_ANDROID_KEY	eas env ✓	eas env ✓	eas env ✓	EAS

Variables marked "—" are either not needed in that EAS environment or are injected at runtime by the Replit build system and must NOT be set in EAS.

Audit notes (May 2026)

- `eas.json` `build.production.env` contains exactly one variable (`EXPO_PUBLIC_API_URL`). No ghost variables were found.
- All 8 `EXPO_PUBLIC_*` variables referenced in source code have a documented injection path above.
- No `EXPO_PUBLIC_*` variables were found that exist only as EAS dashboard entries without a matching code reference.

RevenueCat EAS configuration — verified May 2026

Both `EXPO_PUBLIC_REVENUECAT_IOS_KEY` and `EXPO_PUBLIC_REVENUECAT_ANDROID_KEY` were created in EAS via `eas env:create` for all three environments. Verified with `eas env:list` on 2026-05-08:

```
Environment: development
EXPO_PUBLIC_REVENUECAT_ANDROID_KEY=goog_NhQwCEIvuqrJYSJUqLDCoPjjwIy
EXPO_PUBLIC_REVENUECAT_IOS_KEY=appl_mhkVjHxDzwRmZTeuZSVShSwaimi

Environment: preview
EXPO_PUBLIC_REVENUECAT_ANDROID_KEY=goog_NhQwCEIvuqrJYSJUqLDCoPjjwIy
EXPO_PUBLIC_REVENUECAT_IOS_KEY=appl_mhkVjHxDzwRmZTeuZSVShSwaimi

Environment: production
EXPO_PUBLIC_REVENUECAT_ANDROID_KEY=goog_NhQwCEIvuqrJYSJUqLDCoPjjwIy
EXPO_PUBLIC_REVENUECAT_IOS_KEY=appl_mhkVjHxDzwRmZTeuZSVShSwaimi
```

Note: `EXPO_PUBLIC_REVENUECAT_IOS_KEY` was pre-existing in EAS with the correct value (`appl_mhkVjHxDzwRmZTeuZSVShSwaimi`) across all three environments.

`EXPO_PUBLIC_REVENUECAT_ANDROID_KEY` was newly created for all three environments.

A physical-device build is required to fully validate RevenueCat initialization at runtime (see `E2E_PURCHASE_TEST_GUIDE.md` TC-1). That execution is tracked as follow-up task **#415** (Run the full in-app purchase test on a real device).

Conventions & Preferences

Quick-reference extracted from `replit.md`.

User Preferences

- All `eas` and `expo` commands must be run from `artifacts/knowyourpit/`, never from the workspace root.
- Do not delete the disabled Apple Watch companion app code; it is the starting point for future modernization work.
- Never auto-initiate an EAS build. Always ask the user for confirmation before queuing any build.

Mobile UI Conventions

Keyboard-safe modals

Every modal or bottom sheet that contains a `TextInput` **must** use `AppKeyboardAvoidingView` instead of the raw React Native `KeyboardAvoidingView`. The component lives at `artifacts/knowyourpit/components/AppKeyboardAvoidingView.tsx` and has the correct cross-platform behavior baked in (`"padding"` on iOS, `"height"` on Android).

```
import { AppKeyboardAvoidingView } from "@components/AppKeyboardAvoidingView";

<Modal ...>
  <AppKeyboardAvoidingView style={{ flex: 1 }}>
    { /* modal content with TextInput */ }
  </AppKeyboardAvoidingView>
</Modal>
```

Do **not** write `<KeyboardAvoidingView behavior={Platform.OS === "ios" ? "padding" : "height"}>` inline in new modals — use the wrapper so the behavior stays consistent and reviewable in one place.